SEPIDE: Spatial Execution & Partitioning ISA for Dynamics Evaluation

Nima Sahraneshinsamani
Dept. of Computer Science and Engineering, Universitat Jaume I, Spain
sahrans@uji.es

***Abstract*—Specialized spatial accelerators for local-dynamics workloads (such as cellular automata, discrete physical field simulations, and stencil kernels) rely on *temporal blocking* to alleviate the memory wall. By buffering spatial tiles in on-chip SRAM and computing τ consecutive generations before writing results to DRAM, accelerators amortize off-chip memory traffic. However, in such architectures, temporal depth τ is not an independent loop trip count; it is a *physical coupling variable* that simultaneously dictates halo buffer widths, light-cone dependency footprints, contracting validity windows, and boundary conditions. When hardware control-and-status register (CSR) interfaces expose unconstrained tiling parameters without a verification contract, execution engines suffer from silent data corruption (SDC) caused by buffer truncation, circular address aliasing, or unrefreshed boundary ghost cells.**

In this paper, we present SEPIDE, a virtual ISA and verification contract bridging high-level spatial simulations and capacity-bounded accelerators. SEPIDE enforces seven static obligations (O1–O7), with core light-cone containment and buffer bounds mechanized in Z3 SMT. We prove a Tiling Invariance Theorem guaranteeing bit-exact equivalence to reference semantics for discrete synchronous deterministic dynamics, derive closed-form memory and arithmetic corner taxes for diamond stencils, establish a structural feasibility lower bound ($L_{\max} \geq 1+2r$), and prove per-axis reflection-symmetry commutativity (Lemma 1). Across 69,660 evaluated configurations (151,497 baseline executions) and an independent reference oracle testing 12,252 configurations with 0.0000% false-positive and false-negative rates, SEPIDE eliminates silent data corruption via safe gating and advisory lowering. Dynamic `ADVISORY` lowering reduces off-chip memory traffic by up to $5.38\times$ on small scratchpads ($L_{\max} = 64$) and up to $60.18\times$ on 2D megabyte-scale spatial fabrics ($L_{\max} = 4096$) over spatial-only baselines.

***Index Terms*—Spatial accelerators, temporal blocking, virtual ISA, silent data corruption, formal verification, stencil computation.**

I. Introduction

The performance and energy efficiency of iterative grid computations—such as cellular automata and finite-difference stencils—are fundamentally constrained by off-chip memory bandwidth [1]. Reading and writing the entire grid state to DRAM at every time step requires orders of magnitude more energy than the arithmetic state updates themselves.

To mitigate this memory wall, iterative stencil systems rely on *temporal blocking* (time skewing) [2]–[4]. Concurrently, domain-specific spatial accelerators (e.g., the Groq TSP [5] and Cerebras WSE [6]) demonstrate the power of explicit spatial scheduling and massive on-chip storage for localized execution. In temporal blocking, rather than updating the global domain by a single step $T = 1$, the execution engine loads an on-chip tile of span N' along with an expanded boundary margin (*halo*) into fast SRAM, computes τ consecutive generations locally, and commits the updated interior to memory.

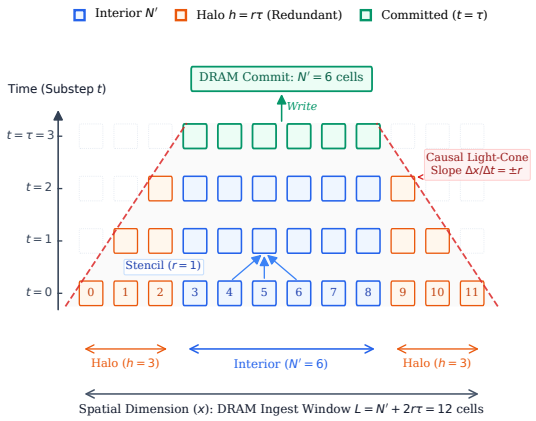


Fig. 1. Geometric coupling in temporal blocking across spatial (x) and temporal (t) dimensions: an ingested window $L = N' + 2r\tau$ contracts at rate $2r$ per substep along the causal light-cone boundary ($\Delta x/\Delta t = \pm r$) to preserve valid interior N' at substep $t = \tau$.

However, an architectural consistency challenge arises at the hardware-software boundary: *temporal depth τ is a tightly coupled physical variable.* As illustrated in Figure 1, advancing a local tile by τ substeps requires a spatial halo of width $h = r \cdot \tau$ (where r is the interaction radius) due to the finite speed of information propagation across the grid (*the causal light cone*). Defining $V(t) = N' + 2r(\tau - t)$ as the valid span after t substeps ($V(0) = L, V(\tau) = N'$), the window evaluated at substep t contracts as $W(t) = V(t+1) = N' + 2r(\tau - 1 - t)$. At step $\tau - 1$, exactly the tile interior N' remains uncorrupted for commit.

When spatial architectures expose temporal depth τ and tile span N' through unconstrained control registers (CSRs), parameter mismatches induce severe semantic divergence. To study these consistency challenges, we formulate an abstract architectural execution model for capacity-bounded spatial scratchpads and evaluate three canonical unverified failure baselines: circular address aliasing (`CSR_WRAP_BUFFER`), halo truncation (`CSR_TRUNCATE_HALO`), and horizon clamping (`CSR_CLAMP_T`). These represent adversarial baseline models rather than physical faults of commercial tapeouts. Crucially, these simulated failures produce no hardware exceptions or

memory faults; they manifest as silent data corruption (SDC), producing incorrect simulation states without warning.

To address this challenge, we introduce SEPIDE (Spatial Execution & Partitioning ISA for Dynamics Evaluation). Rather than delegating physical coupling constraints to unverified software loops or exposing raw hardware CSRs directly, SEPIDE formalizes spatial-temporal execution as an architectural contract between software and silicon.

Key Contributions: (1) **Machine-Independent Virtual ISA:** A portable virtual ISA for local-dynamics workloads supporting explicit stencils, ordered LUT/threshold rules, and exact boundary conditions (`PERIODIC`, `FIXED`, `REFLECT`). (2) **Mechanized Obligations & Tiling Invariance:** Seven static obligations (O1–O7), with core geometric invariants mechanized in Z3 SMT, proving bit-exact reference equivalence under synchronous double-buffered execution. (3) **Dual Memory and Arithmetic Corner Taxes:** Exact closed-form memory ($\mathrm{Tax}_{\mathrm{VN}}^{\mathrm{mem}}$) and arithmetic ($\mathrm{Tax}_{\mathrm{VN}}^{\mathrm{ops}}$) corner taxes incurred when embedding diamond stencils into rectangular SRAM buffers. (4) **Feasibility Bounds & Multi-Scale Advisory Lowering:** A structural feasibility bound ($L_{\max} \geq 1 + 2r$), a discrete majorization proof of balanced pass decomposition dominance, and up to $60.18\times$ memory traffic reduction on 2D megabyte-scale spatial scratchpads ($5.38\times$ on small buffers) over spatial-only baselines. (5) **Reflection Symmetry Attribution & SDC Elimination:** A per-axis reflection-symmetry commutativity theorem (Lemma 1) explaining benchmark bug masking, with safe gating and advisory lowering eliminating silent data corruption across 69,660 configurations.

II. Background and Related Work

A. Local Dynamics and Stencil Operations

Let $\mathcal{G} \subset \mathbb{Z}^d$ be a d-dimensional orthogonal grid with extents $G = (G_1, \ldots, G_d)$. A local-dynamics computation updates cell states $x \in \mathcal{G}$ synchronously across discrete generations $g \in \{0, 1, 2, \ldots\}$. The transition rule is governed by a pure, deterministic function F:

$$x_i^{g+1} = F\big((x_{i+k}^g)_{k \in K}\big) \tag{1}$$

where $K \subset \mathbb{Z}^d$ is an explicit finite stencil of coordinate offsets equipped with a fixed canonical ordering, and $(x_{i+k}^g)_{k \in K}$ denotes the ordered neighborhood state tuple. The interaction radius of the stencil is defined by the Chebyshev (ℓ_∞) norm:

$$r = \max_{k \in K} \|k\|_\infty \geq 1 \tag{2}$$

Common stencils include 1D segments ($K = \{-r, \ldots, r\}$), 2D Moore boxes ($K = \{-r, \ldots, r\}^2$), and 2D von Neumann diamonds ($|u| + |v| \leq r$).

B. The Physical Coupling Variable τ

In temporal blocking, advancing an on-chip tile of span N' for τ consecutive substeps requires an initial window span of $L_i = N_i' + 2r\tau$. As summarized in Table I, the parameter τ physically dictates the required halo width ($h = r\tau$), the contracting active window at substep t ($W_i(t) = N_i' + 2r(\tau - 1 - t)$), redundant compute overhead ($\rho \geq 1$), and amortized off-chip memory traffic (M). In d dimensions, evaluating the expanding halo introduces redundant arithmetic across overlapping tiles:

$$\rho_{\text{tile}} = \frac{\sum_{j=0}^{\tau-1} \prod_{i=1}^{d} (N_i' + 2rj)}{\tau \prod_{i=1}^{d} N_i'} \geq 1.0 \tag{3}$$

While increasing τ reduces memory traffic M, it increases redundant arithmetic ρ, creating an architectural trade-off surface.

TABLE I
Coupled quantities dictated by temporal depth τ.

Architectural Metric	**Mathematical Formulation** ($d = 1$)
Halo Buffer Width	$h = r \cdot \tau$
Ingested Window Span	$L = N' + 2r\tau$
Valid Window at Substep t	$W(t) = N' + 2r(\tau - 1 - t)$
Total Cell Updates per Tile	$C_{\text{updates}} = \sum_{j=0}^{\tau-1} (N' + 2rj)$
Compute Overhead Ratio	$\rho = C_{\text{updates}} / (\tau N') \geq 1$
Off-chip Traffic per Useful Op	$M = (L + N')/(\tau N')$

C. Related Work: Compilers, Accelerators, and Virtual ISAs

Stencil Compilers & Loop Tiling: Extensive research addresses spatial and temporal blocking. Pochoir [3] employs cache-oblivious trapezoids, Halide [7] decouples algorithms from schedules, Devito [8] automates seismic stencils, the Brick library [9] optimizes multi-dimensional vectorization, and Strzodka et al. [4] optimize cache time skewing. While these frameworks optimize general-purpose CPU/GPU hierarchies, they do not target fixed-capacity spatial scratchpads or provide zero-SDC verification contracts.

Spatial Accelerators & Tiling Schemes: Domain-specific architectures such as the Groq TSP [5] and Cerebras WSE [6] utilize explicitly scheduled spatial execution. Diamond tiling [10] and ghost-zone optimization [11] explore spatial-temporal trade-offs. However, on fixed scratchpads, accelerators expose unconstrained configuration registers without a formal load-time contract.

Virtual ISAs & Tile IR Ecosystem: Production compiler ecosystems such as MLIR [12] (via `affine` and `linalg` dialects), Triton-IR [13], CUDA Tile IR, and StableHLO provide structured intermediate representations for tiled matrix and neural compute. However, these IRs lack physical coupling contracts for temporal blocking on scratchpads without virtual memory. SEPIDE introduces this virtual ISA paradigm to local dynamics: it serves as a target-independent contract layer that validates temporal coupling and scratchpad bounds before lowering into hardware-specific PTX, Triton blocks, or spatial control registers.

D. The Vulnerability of Unconstrained CSR Interfaces

In general-purpose processors, memory safety is enforced by MMUs and page-fault handlers. In contrast, specialized spatial accelerators rely on fixed-capacity on-chip SRAM scratchpads without address virtualization. When software configures tile parameters (N, τ) exceeding buffer limits $L_{\max}$, unverified execution engines may fail silently. We model three canonical baseline failure behaviors: address decoders truncating coordinates modulo $L_{\max}$ (`CSR_WRAP_BUFFER`), physical buffers zero-filling out-of-bound halo cells (`CSR_TRUNCATE_HALO`),

or hardware counters silently clamping the temporal horizon (CSR_CLAMP_T), producing silent data corruption without raising errors. SEPIDE replaces ad-hoc register writes with a verified architectural interface.

III. The SEPIDE Virtual ISA Architecture

A. Syntax and Semantic Elements

SEPIDE programs are declared via structured statements: `stmt := DOMAIN shape | STENCIL kind [r] | RULE spec | BOUNDARY mode [val] | PROFILE name | EVOLVE S [T] | RUN`. Key semantic elements include: (1) DOMAIN defines dimension $d \in \{1, 2\}$, extent $G = (G_1, \ldots, G_d)$, and uint8 cell states. (2) STENCIL declares explicit coordinate offsets K, deriving Chebyshev radius $r = \max_{k \in K} \|k\|_\infty$. (3) RULE encodes pure deterministic transition function F via full LUT ($2^{|K|}$ entries for $|K| \leq 9$) or seeded threshold functions. (4) BOUNDARY supports PERIODIC ($x \bmod G$), FIXED (v) ($v \in \{0, 1\}$), or REFLECT (Neumann reflection with period $2G$, where $\text{idx}(x) = m$ if $m < G$ else $2G-1-m$ for $m = x \pmod{2G}$, valid for arbitrary bounces). (5) EVOLVE S [T] advances the global system by S generations under requested pass depth T.

B. Architectural State and the Monoid Law

In SEPIDE, internal tile buffers, halo margins, and intermediate substeps $t \in [1, \tau-1]$ are ephemeral. The *architectural state* of the grid is observed strictly at global generation boundaries g.

Definition 1 (Monoid Law of Evolution)**.** *Sequential evolution in SEPIDE forms an additive monoid under composition with identity* $\text{EVOLVE}(0) \equiv \text{id}$*:* $\text{EVOLVE}(S_1); \text{EVOLVE}(S_2) \equiv \text{EVOLVE}(S_1 + S_2)$*, yielding bit-identical state across all architectural observations.*

To ensure machine-independent verification, architectural grid equivalence is certified for conformance testing via a canonical cryptographic fingerprint: a SHA-256 digest computed over the 64-bit little-endian representations of dimension d, shape tuple G, generation index g, and raw row-major cell byte buffers.

C. Hardware Profiles and Lowering Modes

SEPIDE abstracts hardware into two capability profiles: **Profile A (`V1_MONOLITHIC_1024`)** models single-pass evaluation over grids with $|\mathcal{G}| \leq 1024$ without spatial tiling ($\rho \equiv 1.0$). **Profile B (`TILE_WINDOW`)** models scratchpad execution constrained by buffer limit $L_{\max}$. For rectangular tiles $N' = (N'_1, \ldots, N'_d)$, scratchpad capacity requires $N'_i + 2r\tau \leq L_{\max}$ along every axis i, yielding:

$$T_{\max} = \min_{i=1}^{d} \left\lfloor \frac{L_{\max} - N'_i}{2r} \right\rfloor = \left\lfloor \frac{L_{\max} - \max_i N'_i}{2r} \right\rfloor \quad (4)$$

where $N'_i = \min(N_i, G_i)$. Lowering to Profile B operates in **EXACT Mode** (strictly executing depth T_{req} and rejecting prior to memory mutation if $T_{\text{req}} > T_{\max}$) or **ADVISORY Mode** (dynamically choosing legal spatial tile spans $N' \leq N$ and balanced pass horizons $\tau_p \leq T_{\max}$ summing to S, weakly dominating greedy pass decompositions in arithmetic overhead).

D. Verification Obligations and Tiling Invariance

A conforming SEPIDE target statically enforces seven obligations (Table II): O1 (*Bind-Before-Mutate*) prevents state mutation on illegal plans; O2 (*Well-Formedness*) checks extents ($G_i, N'_i, S \geq 1$); O3 (*Radius Consistency*) enforces exact equality $r_{\text{decl}} = \max_{k \in K} \|k\|_\infty$; O4 (*Capacity Bounds*) guarantees $N'_i + 2h_p \leq L_{\max}$ (with $h_p \geq r\tau_p$, taking minimal halo $h_p = r\tau_p$); O5a (*Halo Sufficiency*) requires halo width $h_p \geq r\tau_p$; O5b (*Boundary Refresh*) mandates ghost synchronization before each substep $t \geq 1$ under FIXED and REFLECT (unless waived by Lemma 1); O6 (*Partition & Depth Conservation*) partitions $\mathcal{G}$ and conserves $\sum \tau_p = S$; and O7 (*Rule Purity*) requires deterministic transitions.

Regarding O3, exact equality $r_{\text{decl}} = r$ serves as an interface canonicalization contract: while under-declaration ($r_{\text{decl}} < r$) starves the causal light cone, over-declaration ($r_{\text{decl}} > r$) under standard symmetric $2r$-contracting kernels leaves residual halo margin, misaligning fixed commit offsets relative to interior I.

Theorem 1 (Tiling Invariance)**.** *Let $P = (G, K, F, \text{BC}, S)$ be a local-dynamics program with pure transition rule F. Let a lowering plan consist of pass depths $\tau_1, \ldots, \tau_q$ with $\sum_{p=1}^{q} \tau_p = S$ and spatial tile partitions satisfying obligations O2–O6. Assuming double-buffered synchronous state updates (tile loads read immutable generation g and commit to $g + \tau_p$ without intra-pass races), tiled execution is bit-identical to mathematical reference semantics for discrete deterministic dynamics:*

$$\Phi^S_{\text{tiled}}(X_0) \equiv \Phi^S(X_0) \quad (5)$$

Proof Sketch: Let E_g denote the boundary-extended global grid state at generation g. For each pass p of depth τ_p, we show by induction on $t \in [0, \tau_p]$ that every site in the contracting window $I \oplus B_\infty(r(\tau_p - t))$ holds the exact reference state E_{g+t}. For $t = 0$, this holds by the initial buffer load (O4, O5a). For step $t \to t+1$, each site $c \in I \oplus B_\infty(r(\tau_p - (t+1)))$ evaluates neighbors $c + k$ with $k \in K$. By the Chebyshev radius bound $\|k\|_\infty \leq r$ (O3), every neighbor lies within $I \oplus B_\infty(r(\tau_p - t))$ and hence holds E_{g+t}. Purity of F (O7) guarantees $x_c^{g+t+1} = E_{g+t+1}(c)$. Ghost sites on domain boundaries are restored at each substep by the boundary condition (O5b), or are preserved by Lemma 1 under reflection symmetry. At $t = \tau_p$, the tile interior I holds $E_{g+\tau_p}$. Because spatial tile interiors partition the global domain $\mathcal{G}$ without gaps or overlaps (O6), committing I updates the global generation to $g + \tau_p$. Induction over passes $p \in \{1, \ldots, q\}$ with $\sum \tau_p = S$ completes the proof. □

Mechanized SMT Verification of Obligations: To substantiate Theorem 1 beyond hand-written induction, we mechanized obligations O1–O6 in the Z3 SMT solver [14]. For arbitrary $N \geq 1, \tau \geq 1, r \geq 1$, Z3 proves that dependency escape from halo $[-h, N - 1 + h]$ is UNSAT under $h \geq r\tau$, while light-cone starvation under $h < r\tau$ yields a constructive SAT witness. Furthermore, Z3 proves that buffer address violation

TABLE II
OBLIGATION STATIC ENFORCEMENT TEST COVERAGE.

Obl.	Property	Negative Mutant / Invariant Check
O1	Bind-Before-Mutate	Mutation prevented on illegal plan
O2	Well-Formedness	Grid dimension zero ($G_0 = 0$) rejected
O3	Radius Consistency	Radius mismatch ($r_{\text{decl}} \neq r$) rejected
O4	Capacity Bounds	Window span exceeding $L_{\max}$ rejected
O5a	Halo Sufficiency	Buffer width $h_p < r\tau_p$ rejected
O5b	Boundary Refresh	Unrefreshed ghost cells on `FIXED` rejected
O6	Tiling Partition	Step count conservation ($\sum \tau_p \neq S$) rejected
O7	Rule Purity	Stateful non-deterministic rule rejected

(addr $\geq L_{\max}$) is `UNSAT` iff $N' + 2r\tau \leq L_{\max}$ (O4), and verifies generation bijection under O6. Obligation O1 (*Bind-Before-Mutate*) enforces these constraints at load time, rejecting invalid programs before state mutation.

IV. Analytical Modeling and Correctness Proofs

A. Bounding-Box Corner Taxes for Diamond Stencils

When spatial accelerators evaluate 2D local dynamics, rectangular on-chip SRAM buffers store square windows of span $L \times L$ (where $L = N' + 2R$ and $R = r\tau$). For Moore box stencils, the light-cone footprint is naturally square, filling the entire buffer. However, for von Neumann stencils (L_1 diamonds), allocating a rectangular bounding box loads and computes cells outside the true dependency cone. We distinguish between the *memory ingest tax* and the *arithmetic operation tax*.

Theorem 2 (Dual Corner Taxes for von Neumann Diamonds)**.** *Let an $N' \times N'$ tile interior evolve for τ substeps under a 2D von Neumann stencil of radius r with reach $R = r\tau$. (1)* ***Memory Ingest Corner Tax:*** *In the allocated $(N' + 2R)^2$ bounding box loaded from off-chip DRAM, exactly $2R(R+1)$ cells lie outside the initial dependency cone. The memory ingest tax fraction is:*

$$\text{Tax}_{\text{VN}}^{\text{mem}}(N', r, \tau) = \frac{2R(R+1)}{(N'+2R)^2}, \quad R = r\tau \tag{6}$$

(2) ***Arithmetic Operation Corner Tax:*** *Over τ contracting substeps, the active window at substep $j \in \{0, \ldots, \tau-1\}$ contracts to $N' + 2r(\tau - 1 - j)$. Letting $m = \tau - 1 - j$ denote the remaining reach, exactly $2rm(rm+1)$ redundant corner cell updates occur at substep j. The cumulative arithmetic operation tax fraction is:*

$$\text{Tax}_{\text{VN}}^{\text{ops}}(N', r, \tau) = \frac{\sum_{m=0}^{\tau-1} 2rm(rm+1)}{\sum_{m=0}^{\tau-1}(N'+2rm)^2} \tag{7}$$

Proof Sketch: Tile interior coordinates are $I = [0, N'-1]^2$. The true dependency footprint after τ steps is the ℓ_1 ball $I \oplus K^{\oplus\tau} = \{(y,x) \mid \text{dist}_y(y, I) + \text{dist}_x(x, I) \leq R\}$. For (1), non-influencing cells in the $(N'+2R)^2$ box lie in the four $R \times R$ corners where $\text{dist}_y \geq 1, \text{dist}_x \geq 1$ and $\text{dist}_y + \text{dist}_x > R$. Each corner excludes $\sum_{i=1}^{R}(R-i+1) = \frac{R(R+1)}{2}$ cells, giving $2R(R+1)$ unneeded DRAM reads. For (2), at substep j with remaining reach $m = \tau - 1 - j$, exactly $2rm(rm+1)$ corner cells are redundantly updated out of $(N'+2rm)^2$ active evaluations. Summing over all substeps yields $\text{Tax}_{\text{VN}}^{\text{ops}}$. □

Figure 2 validates these closed-form formulations against exact geometric simulation counters. For small tiles ($N' = 8, r = 1, \tau = 2 \implies R = 2$), the memory ingest tax is 8.33% ($\frac{12}{144}$), whereas the arithmetic operation tax is only 2.44% ($\frac{4}{164}$). Both taxes asymptotically vanish ($O(R^2/N'^2)$) as $N' \to \infty$.

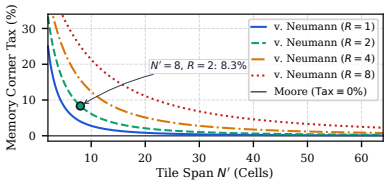


Fig. 2. Bounding box corner tax for von Neumann diamond stencils across tile spans N' and temporal reaches $R = r\tau$.

B. Structural Feasibility and Advisory Lowering

When an `EXACT` configuration (N, T) violates capacity obligation O4, can the program be safely repaired?

Definition 2 (Target Feasibility Lower Bound)**.** *A hardware buffer limit $L_{\max}$ is feasible for stencil radius r if and only if $L_{\max} \geq 1 + 2r$. If $L_{\max} < 1 + 2r$, no interior cell can complete even a single substep without exceeding capacity; the target is structurally infeasible.*

When $L_{\max} \geq 1 + 2r$, SEPIDE guarantees the existence of a valid execution binding via a universal joint repair: $(N_{\text{joint}}, T_{\text{joint}}) = (L_{\max} - 2r, 1)$, serving as a constructive feasibility certificate.

Dominance of Balanced Decomposition: To execute S evolution steps under effective capacity limit $T_{\text{eff}} \leq T_{\max}$, the verifier partitions S across $q = \lceil S/T_{\text{eff}} \rceil$ passes. A naive greedy decomposition executes $\lfloor S/T_{\text{eff}} \rfloor$ passes of depth T_{eff} followed by a remainder pass. In contrast, SEPIDE's `ADVISORY` planner balances pass depths: $\text{bal}(S, T_{\text{eff}}) = (b+1, \ldots, b+1, b, \ldots, b)$ with $b = \lfloor S/q \rfloor$, where $S \bmod q$ passes have depth $b+1$ (e.g., $(5,5)$ vs. $(6,4)$ for $S = 10, T_{\text{eff}} = 6$). For fixed tile interior N' in $d \in \{1, 2\}$, cell updates per pass are $A_d(\tau) = \sum_{j=0}^{\tau-1}(N' + 2rj)^d$ and I/O traffic is $B_d(\tau) = (N' + 2r\tau)^d + (N')^d$. Under cost $C_d(\tau) = A_d(\tau) + \beta B_d(\tau)$ ($\beta > 0$), whole-plan cost is $J_{\text{plan}} = \frac{1}{S(N')^d}\sum_{p=1}^{q} C_d(\tau_p)$. Marginal increments $\Delta A_d(\tau) = (N' + 2r\tau)^d$ strictly increase in τ, while $\Delta B_d(\tau)$ is non-decreasing ($2r$ in 1D; strictly increasing in 2D). Thus pass cost $C_d(\tau)$ is discretely strictly convex ($\Delta^2 C_d > 0$). For any two pass depths with $a \geq b + 2$, strictly increasing differences $\Delta C_d(a-1) > \Delta C_d(b)$ directly yield $C_d(a-1) + C_d(b+1) < C_d(a) + C_d(b)$. Transferring substeps from deeper to shallower passes strictly minimizes total cost, establishing $J_{\text{bal}} \leq J_{\text{greedy}}$, with strict dominance whenever greedy is unbalanced.

Feasibility Certificate vs. Advisory Lowering (Multi-Scale): While the minimal feasibility certificate $(L_{\max} - 2r, 1)$ guarantees satisfiability, executing S passes of depth 1 incurs heavy memory traffic. In contrast, our `ADVISORY` planner maximizes locality via balanced decomposition. On small scratchpads ($G = 128, r = 1, S = 12, L_{\max} = 64$), the certificate requires 12 passes transferring 3,144 bytes, whereas the ADVISORY plan executes in 2 balanced passes of depth 6 transferring 584 bytes—a $5.38\times$ I/O reduction. On spatial fabrics with megabyte-scale SRAM (Groq TSP [5], Cerebras

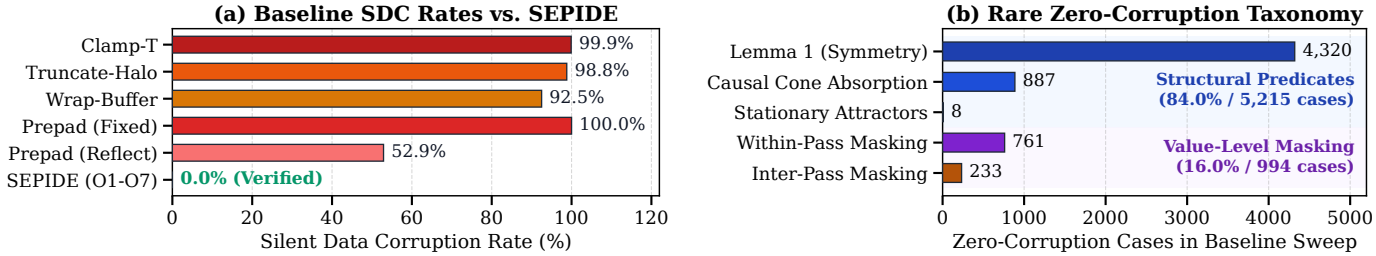


Fig. 3. (a) Silent Data Corruption rates across unverified hardware baselines vs. SEPIDE. (b) Rigorous taxonomy of rare zero-corruption occurrences in baseline execution engines.

WSE [6]), scaling $L_{\max}$ to 2048 and 4096 on large 2D grids ($G = 2048^2, 4096^2$, spanning 16–32 MB on-chip buffers) yields $43.80\times$ and $60.18\times$ traffic reductions over spatial-only baselines ($T = 1$).

C. Per-Axis Reflection-Symmetry Commutativity

A critical source of potential boundary corruption in spatial accelerators is omitting the ghost cell refresh before intermediate substeps $t \geq 1$ (`CSR_PREPAD_ONCE`). However, we prove that for rules exhibiting per-axis reflection symmetry, intermediate refresh is mathematically redundant.

Lemma 1 (Per-Axis Reflection Symmetry Commutativity)**.** *Let $\mathcal{G} \subset \mathbb{Z}^d$. For each axis i, let σ_i denote reflection across an axis-i boundary, and $\hat{\sigma}_i$ the induced neighborhood permutation. If: (1) $k \in K \iff \sigma_i(k) \in K$, and (2) $F(\hat{\sigma}_i \nu) = F(\nu)$ for all neighborhoods ν, then synchronous evolution Φ preserves even symmetry across every boundary. Thus, intermediate ghost refresh (O5b) is redundant under `REFLECT`, and single pre-padding (`CSR_PREPAD_ONCE`) is bit-exact for all $\tau \geq 1$.*

Proof Sketch: For ghost c with mirror $c' = \sigma_i(c)$, K's invariance gives $\mathcal{N}(c) = \hat{\sigma}_i \mathcal{N}(c')$. Even symmetry and F's commutativity imply $F(\mathcal{N}(c)) = F(\hat{\sigma}_i \mathcal{N}(c')) = F(\mathcal{N}(c'))$. The halo evolves identically to the reflected interior, proving the claim by induction on $t \in [1, \tau]$. □

Benchmark Attribution and Insufficiency of Point Symmetry: Lemma 1 provides a structural sufficient condition explaining why certain unverified baselines avoid corruption. Crucially, per-axis reflection ($k_i \mapsto -k_i$) is required; point inversion ($k \mapsto -k$) is insufficient. For instance, the 2D von Neumann rule $F = c \oplus (n \wedge e) \oplus (s \wedge w)$ is point-symmetric but violates axis reflection; under `REFLECT` ($\tau = 2$, 5×5), unrefreshed pre-padding corrupts 83% of random states ($k = 100$). In 1D, asymmetric Rule 30 corrupts 25% of states. Full per-axis symmetric rules (Rule 90, Life) compute exact unrefreshed trajectories across all tested states. Evaluating accelerators exclusively on symmetric benchmarks systematically masks boundary bugs.

V. Experimental Setup and Evaluation

Workload Setup: Our evaluation suite spans four axes: (1) seven rules (Wolfram ECA 30, 90, 110 [15], asymmetric radius-2 1D, Game of Life [16], asymmetric 2D VN, asymmetric 2D Moore $r = 2$); (2) four boundary modes (`PERIODIC`, `FIXED(0)`, `FIXED(1)`, `REFLECT`); (3) ragged grid geometries $G \in \{(8,), (10,), (13,), (8,8), (9,11), (11,9), (128,)\}$; and (4) scratchpad capacities $L_{\max} \in [1, 64]$.

TABLE III
Baseline Silent Corruption Breakdown by Transition Rule.

Rule	Evals	Struct. 0s	Value Mask.	Corrupt. Rate
ECA 30	16,776	78	49	99.2%
ECA 90	16,776	1,886	345	86.7%
ECA 110	16,776	78	132	98.7%
ECA r2 asym	22,560	209	105	98.6%
Life (2D)	23,508	2,629	112	88.3%
VN r1 asym	23,508	109	142	98.9%
Moore r2 asym	31,593	226	109	98.9%
Baseline Total	**151,497**	**5,215**	**994**	**95.9%**
SEPIDE (All)	**69,660**	**Gated / Rebound**		**0.0%**

Experiment 1 (SDC Elimination and Oracle): Across 69,660 unique tiling configurations (151,497 baseline executions, $k = 8$ seeds), unverified hardware engines exhibit severe silent corruption (Figure 3(a)): 99.9% under horizon clamping (`CSR_CLAMP_T`), 98.8% under halo truncation (`CSR_TRUNCATE_HALO`), 92.5% under circular buffer aliasing (`CSR_WRAP_BUFFER`), and 100.0% under single pre-padding (`CSR_PREPAD_ONCE`) on `FIXED`. Under `REFLECT`, `CSR_PREPAD_ONCE` averages 52.9% corruption: exactly 0.0% on symmetric rules (Rule 90, Life, Lemma 1), and 75.0% on asymmetric rules. In contrast, SEPIDE eliminates silent data corruption across all 69,660 configurations via safe load-time gating (O1) and advisory lowering.

Independent Reference Oracle Verification: We evaluated verifier decisions against an independent brute-force reference oracle over 12,252 generated configurations ($d \in \{1, 2\}$, non-square tiles, ragged extents, $L_{\max} \in [8, 96]$). The verifier achieved complete agreement with the oracle: a **0.0000% false-positive rate** (0 rejected out of 9,585 safe) and a **0.0000% false-negative rate** (0 accepted out of 2,667 corrupting). While finite sampling cannot universally establish completeness, this extensive agreement alongside Theorem 1 confirms that O1–O7 establish an exact, sound boundary contract.

Figure 3(b) and Table III break down the 6,209 zero-corruption baseline cases. 5,215 cases (84.0%) stem from pre-execution structural predicates: *Reflection Symmetry* (4,320 cases, Lemma 1); *Causal Cone Absorption* (887 cases, where $W_{\text{base}} \neq W_{\text{true}}$ never intersects interior N'); and *Stationary Attractors* (8 cases reaching fixed points). The remaining 994 cases (16.0%) stem from post-hoc Boolean value masking (761 within-pass, 233 inter-pass).

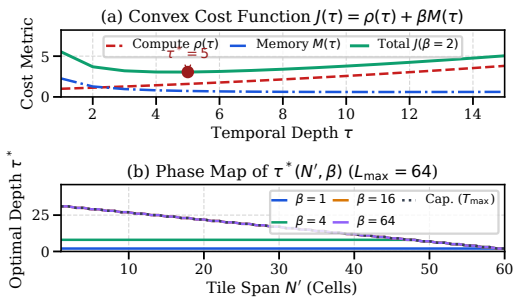


Fig. 4. (a) Convex cost function $J(\tau) = \rho(\tau) + \beta M(\tau)$ revealing unique optimal depth τ^*. (b) Phase map of optimal depth $\tau^*(N, \beta)$ under buffer limit $L_{\max} = 64$.

Experiment 2 (Conformance and Monoid Invariance): Experiment 2 verifies semantic conformance against golden models across 38 test cases (100% bit-exact SHA-256 matches). Testing the Monoid Law ($\text{EVOLVE}(S_1 + S_2) \equiv \text{EVOLVE}(S_1); \text{EVOLVE}(S_2)$) across all 16 combinations of dimension, boundary condition, and mode on ragged grids confirms zero bit differences, while closed-form cost models match per-cell simulation counters with zero deviation.

Experiment 3 (Phase Diagram and Optimal Depth): We evaluate the trade-off between memory traffic $M(\tau)$ and compute overhead $\rho(\tau)$ across 1,872 configurations (11,232 points).

As shown in Figure 4(a), the combined cost metric $J(\tau) = \rho(\tau) + \beta M(\tau)$ is strictly convex in τ, where β denotes the memory-to-compute penalty ratio. At small τ, memory traffic dominates; at large τ, halo recomputation explodes. The optimal depth τ^* balances this trade-off until clipped by the physical capacity bound $T_{\max} = \lfloor (L_{\max} - \max_i N_i')/(2r) \rfloor$. Figure 4(b) maps optimal depth across tile spans N and ratios β; our analytical 1D and 2D minimizers match discrete simulations in 100% of evaluated cases.

VI. Discussion and Conclusion

Backend Portability (GPUs and NPUs): Although evaluated on abstract spatial scratchpads, SEPIDE maps directly to commercial hardware: (1) *GPUs:* Ingested tile windows map to Shared Memory, temporal passes map to cooperative thread blocks in CUDA or Triton [13], and $L_{\max}$ reflects per-SM shared memory. (2) *NPUs:* Tiles map to local SRAM in fabrics like Google TPU or Groq TSP [5], where $T_{\max}$ limits un-spilled allocations. SEPIDE acts as a target-independent contract layer, validating spatial-temporal coupling invariants before hardware emission.

Agent-Facing Virtual ISAs and Late Binding: AI agents and compilers increasingly synthesize accelerator configurations. Exposing raw CSRs is hazardous: agents routinely propose mismatched parameters or omit boundary refresh, causing silent data corruption without hardware faults. SEPIDE resolves this as an *agent-facing virtual ISA*: agents declare algorithmic intent (`DOMAIN`, `STENCIL`, `RULE`, `BOUNDARY`, `EVOLVE`), while the verifier statically enforces O1–O7 with actionable diagnostic witnesses. Through *late binding*, SEPIDE decouples algorithmic intent from physical SRAM capacity ($L_{\max}$).

Limitations: We identify three limitations: (1) *Discrete semantics:* SEPIDE targets discrete dynamics; continuous PDEs require bounding floating-point drift. (2) *Tiling scope:* We focus on scratchpad blocking; streaming arrays require flow control. (3) *Fault modeling:* Baseline corruption reflects architectural register models rather than physical silicon defects.

Conclusion: We presented SEPIDE, a virtual ISA and verification contract for spatial accelerators. Governed by obligations O1–O7 mechanized in Z3 SMT, SEPIDE eliminates silent data corruption across 69,660 configurations via safe gating and advisory lowering, verified with 0.0000% false-positive/negative rates against an independent reference oracle. Our closed-form models characterize dual corner taxes, structural feasibility ($L_{\max} \geq 1 + 2r$), and reflection commutativity. Dynamic advisory lowering achieves up to $60.18\times$ memory traffic reduction on 2D megabyte-scale fabrics ($5.38\times$ on small scratchpads), establishing a robust contract for spatial computing.

References

[1] M. Horowitz, "1.1 Computing's energy problem (and what we can do about it)," in *IEEE ISSCC Dig. Tech. Papers*, 2014, pp. 10–14.

[2] G. Wellein, G. Hager, T. Zeiser, M. Wittmann, and H. Fehske, "Efficient temporal blocking for stencil computations by multicore-aware wavefront parallelization," in *Proc. IEEE COMPSAC*, 2009, pp. 579–586.

[3] Y. Tang, R. A. Chowdhury, B. C. Kuszmaul, C.-K. Luk, and C. E. Leiserson, "The Pochoir stencil compiler," in *Proc. ACM SPAA*, 2011, pp. 117–128.

[4] R. Strzodka, M. Shaheen, D. Pajak, and H.-P. Seidel, "Cache accurate time skewing in iterative stencil computations," in *Proc. ICPP*, 2011, pp. 571–581.

[5] D. Abts, J. Ross, J. Sparling, M. Wong-VanHaren, M. Baker, T. Hawkins, A. Bell, J. Thompson, T. Kahsai, G. Kimmell *et al.*, "Think fast: A tensor streaming processor (TSP) for accelerating deep learning workloads," in *Proc. ACM/IEEE ISCA*, 2020, pp. 145–158.

[6] S. Lie, "Cerebras architecture deep dive: First look inside the hardware/software co-design for deep learning," *IEEE Micro*, vol. 43, no. 3, pp. 18–30, 2023.

[7] J. Ragan-Kelley, C. Barnes, A. Adams, S. Paris, F. Durand, and S. Amarasinghe, "Halide: A language and compiler for optimizing parallelism, locality, and recomputation in image processing pipelines," in *Proc. ACM PLDI*, 2013, pp. 519–530.

[8] F. Luporini, M. Louboutin, M. Lange, N. Kukreja, P. Witte, J. Hückelheim, C. Yount, P. H. J. Kelly, F. J. Herrmann, and G. J. Gorman, "Architecture and performance of Devito, a system for automated stencil computation," *ACM TOMS*, vol. 46, no. 1, pp. 1–28, 2020.

[9] T. Zhao, P. Basu, M. Hall, and D. Mu, "Exploiting reuse and vectorization in stencil computations on AVX-512 and GPUs with the brick library," in *Proc. ACM/IEEE SC*, 2018, pp. 220–232.

[10] U. Bondhugula, V. Bandishti, and I. Pananilath, "Diamond tiling: Tiling techniques to maximize parallelism for stencil computations," *IEEE TPDS*, vol. 28, no. 5, pp. 1285–1298, 2017.

[11] J. Meng and K. Skadron, "Performance modeling and automatic ghost zone optimization for iterative stencil loops on GPUs," in *Proc. ACM ICS*, 2009, pp. 256–265.

[12] C. Lattner, M. Amini, U. Bondhugula, A. Cohen, A. Davis, J. Pienaar, R. Riddle, T. Shpeisman, N. Vasilache, and O. Zinenko, "MLIR: Scaling compiler infrastructure for domain specific architectures," in *Proc. IEEE/ACM CGO*, 2021, pp. 2–14.

[13] P. Tillet, H. T. Kung, and D. Cox, "Triton: An intermediate language and compiler for tiled neural network computations," in *Proc. ACM MAPL*, 2019, pp. 10–19.

[14] L. De Moura and N. Bjørner, "Z3: An efficient SMT solver," in *Proc. TACAS*, 2008, pp. 337–340.

[15] S. Wolfram, *A New Kind of Science*. Wolfram Media, 2002.

[16] M. Gardner, "Mathematical games: The fantastic combinations of John Conway's new solitaire game 'Life'," *Scientific American*, vol. 223, no. 4, pp. 120–123, 1970.